Claude Code Guardrails

Five safety checks for Claude Code, explained

by Gabrielė · Codeart (codeart.lt)

The problem

An agent that can run arbitrary shell commands will, sooner or later, try to run one you did not mean to approve. Guardrails catch that before it executes — not after.

What’s inside

- A .claude/settings.json wiring up three hook registrations
- A guard.sh script implementing five distinct safety checks
- This guide
- Drop-in setup — one dependency (jq), no build step

How it’s wired

Claude Code hooks are shell commands the CLI runs before or after a tool call, fed a JSON payload on stdin. guard.sh reads that payload once and runs five checks against it. A PreToolUse hook can block the call outright (exit code 2); a PostToolUse hook can only observe, so it’s used here for the audit log.

`.claude/settings.json`

```
{
  "hooks": {
    "PreToolUse": [
      {
        "matcher": "Bash",
        "hooks": [
          { "type": "command", "command": "$CLAUDE_PROJECT_DIR/.claude/guard.sh" }
        ]
      },
      {
        "matcher": "Write|Edit|MultiEdit",
        "hooks": [
          { "type": "command", "command": "$CLAUDE_PROJECT_DIR/.claude/guard.sh" }
        ]
      }
    ],
    "PostToolUse": [
      {
        "matcher": "Bash",
        "hooks": [
          { "type": "command", "command": "$CLAUDE_PROJECT_DIR/.claude/guard.sh" }
        ]
      }
    ]
  }
}
```

Three hook registrations, five checks — guard.sh looks at hook_event_name and tool_name to decide which checks apply to this call.

1 · Destructive filesystem commands

Blocks rm -rf aimed at /, ~, or the current directory, raw disk writes via dd, and fork-bomb patterns. These are the commands where there is no undo.

2 · Force-push and history rewrites on protected branches

Blocks git push --force (or -f) specifically when the target is main or master, plus filter-branch and force-pushing --all. Force-pushing a feature branch you own is still allowed — the check only fires on shared history.

3 · Skipping safety checks

Blocks --no-verify and --no-gpg-sign (both exist to skip a check someone deliberately set up), chmod 777, and piping a remote script straight into a shell (curl ... | bash, wget ... | sh) — read a script before it runs, always.

4 · Writes to sensitive files

Blocks Write/Edit/MultiEdit targeting .env, SSH keys, .aws/credentials, credentials.json, or .git/config. If a credential needs to change, do it by hand — an agent shouldn’t be the one touching it.

5 · Audit log

Every Bash command that runs — blocked or not — gets one line in .claude/guard.log with a timestamp. It never blocks anything; it just means there’s always a record, even for the commands the other four checks didn’t catch.

The full guard.sh script

.claude/guard.sh

```
#!/usr/bin/env bash
# Claude Code Guardrails -- guard.sh
# Reads a PreToolUse/PostToolUse hook payload from stdin and decides
# whether to allow, block, or just log the tool call.
#
# Exit codes (per Claude Code's hook contract):
#   0 = allow (stdout/stderr shown to user only)
#   2 = block (stderr is fed back to Claude as the reason)

set -euo pipefail

payload="$(cat)"
tool_name="$(echo "$payload" | jq -r '.tool_name // empty')"
command="$(echo "$payload" | jq -r '.tool_input.command // empty')"
file_path="$(echo "$payload" | jq -r '.tool_input.file_path // empty')"
hook_event="$(echo "$payload" | jq -r '.hook_event_name // empty')"

log_dir="$(dirname "$0")"
log_file="$log_dir/guard.log"

block() {
  echo "Guardrails blocked this: $1" >&2
  exit 2
}

# 1: destructive filesystem commands
check_destructive_fs() {
  case "$command" in
    *"rm -rf /"*|*"rm -rf ~"*|*"rm -rf \$HOME"*|*"rm -rf ."*)
      block "destructive rm -rf targeting root, home, or the current directory." ;;
    *"dd if="*"of=/dev/"*)
      block "raw disk write via dd -- this can destroy a whole disk." ;;
    *":(){ :|:& };:"*)
      block "fork bomb pattern detected." ;;
  esac
}

# 2: force-push / history rewrite on protected branches
check_protected_branch() {
  case "$command" in
    *"push"*"--force"*|*"push -f"*)
      if echo "$command" | grep -qE '(^|[[:space:]])(main|master)([[:space:]]|$)'; then
        block "force-push to main/master."
      fi ;;
    *"filter-branch"*|*"push"*"--force"*"--all"*)
      block "history-rewriting operation (filter-branch / force-push --all)." ;;
  esac
}
```

```
# 3: skipping safety checks
check_skip_safety() {
  case "$command" in
    *"--no-verify"*|*"--no-gpg-sign"*)
      block "a flag that skips commit hooks or signature verification." ;;
    *"chmod 777"*|*"chmod -R 777"*)
      block "chmod 777 -- overly permissive file permissions." ;;
    *"curl"*"| bash"*|*"curl"*"|bash"*|*"wget"*"| sh"*|*"wget"*"|sh"*)
      block "piping a remote script straight into a shell -- read it first." ;;
  esac
}

# 4: writes to sensitive files
check_sensitive_write() {
  case "$file_path" in
    *".env"*|*"id_rsa"*|*"id_ed25519"*|*".aws/credentials"*|*".ssh/"*| \
    *"credentials.json"*|*".git/config")
      block "write to a sensitive/credentials path ($file_path). Edit it by hand instead." ;;
  esac
}

# 5: audit log (never blocks, just records)
write_audit_log() {
  printf '%s\t%s\t%s\n' "$(date -Iseconds)" "$tool_name" "${command:-$file_path}" >> "$log_file"
}

case "$hook_event" in
  PreToolUse)
    if [ "$tool_name" = "Bash" ]; then
      check_destructive_fs
      check_protected_branch
      check_skip_safety
    fi
    if [ "$tool_name" = "Write" ] || [ "$tool_name" = "Edit" ] || [ "$tool_name" = "MultiEdit" ];
then
      check_sensitive_write
    fi
    ;;
  PostToolUse)
    write_audit_log
    ;;
esac

exit 0
```

Setup

1. Create a .claude folder in your project root if it doesn’t exist.
2. Save the settings.json above as .claude/settings.json.
3. Save the guard.sh script above as .claude/guard.sh.
4. Make it executable: chmod +x .claude/guard.sh
5. Make sure jq is installed (brew install jq / apt install jq).
6. Restart Claude Code so it picks up the new settings.

Free resource by Gabrielė · Codeart — more at codeart.lt/en/resources. Questions, or found an edge case these checks miss? Reach out via the socials linked on the site.